PDF ③ 그림 중심 이해·적용·실습 교재 (Visual Understanding & Practice)

대상 범위: SSAFY DB — SQL 01 (153p) · SQL 02 (113p) · Many To One Relationships 01 (81p) · Many To One Relationships 02 (76p)

설계 구조: **PART A (외우기보다 이해해서 적용해야 하는 내용 — 설명형 정밀 다이어그램 10종 & 3단계 실습) + PART B (외워두면 좋은 내용 — 명령어·공식·패턴 & 회상 훈련)**

0. 이 교재의 학습 설계 원칙: 왜 PART A와 PART B로 나누었는가?

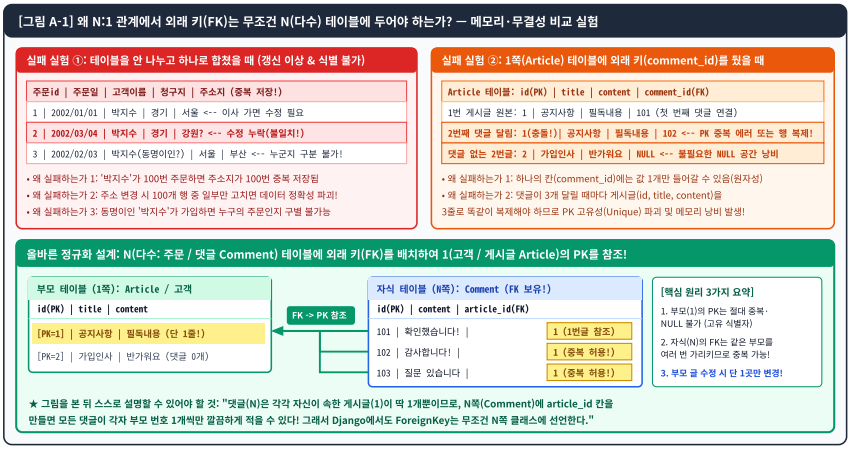
데이터베이스와 Django ORM을 공부할 때 모든 내용을 똑같이 암기하려고 하면 실패한다. 예를 들어 `WHERE` 절에 왜 `AVG()`를 쓸 수 없는지, `LEFT JOIN`에서 왜 `WHERE articles.userId IS NULL`이 '글을 안 쓴 회원'을 찾아주는지, 왜 `CommentForm`에서 `article`을 빼고 `save(commit=False)`를 호출하는지는 **내부 데이터 상태 변화(U: 이해 중심)를 눈으로 보아야** 새로운 문제에 적용(P)할 수 있다. 반면 SQL 4대 분류나 `ALTER TABLE` 구문, 데코레이터 이름은 **정확한 표기와 조건(K: 기억 중심)을 외워두면** 코딩 속도가 비약적으로 빨라진다. 본 교재는 이를 명확히 분리하여 설계하였다.

PART A. 외우기보다 이해해서 적용해야 하는 내용 (원리 · 설명형 그림 · 3단계 실습)

PART A의 모든 주제는 [문제 상황 → 왜 어려운가(오개념) → 핵심 원리 → 구체적 입력·상태 변화를 담은 설명형 그림 → 3단계 실습(따라 하기 / 일부 변경하기 / 스스로 적용하기) → 결과 및 변형 문제]의 흐름으로 진행된다.

Unit A-1. 동명이인·중복 저장 문제와 PK·FK 테이블 분리 및 N:1 외래 키 배치 원리 [C02, C14, C15]

- **1) 문제 상황 및 왜 어려운가(초보자 오개념)**: 게시글(`Article`, 1)과 댓글(`Comment`, N) 관계를 설계할 때, 초보자는 직관적으로 *"게시글이 댓글들을 가지고 있으니까 `Article` 테이블에 `comment_id`(외래 키) 컬럼을 넣으면 되지 않을까?"*라고 생각한다.
- **2) 핵심 원리**: 만약 1을 담당하는 `Article` 테이블에 외래 키를 두면, 1번 게시글에 댓글이 3개 달릴 때 1번 게시글의 `id`, `title`, `content`를 3번 중복 저장해야 하며(PK 중복 불가 제약 위반 혹은 데이터 중복 낭비), 게시글 제목을 수정할 때 3개 행을 모두 고쳐야 한다. 반대로 **N을 담당하는 `Comment` 테이블에 외래 키(`article_id`)를 두면**, 게시글은 단 1행만 유지되고 댓글이 생성될 때마다 부모 게시글의 PK(1)만 가리키므로 데이터 중복이 0이 된다!



[그림 A-1] 단일 테이블(실패 ①) vs 1쪽에 FK 배치(실패 ②) vs N쪽에 FK 배치(정규화 성공)의 실제 행 데이터 비교 (원본 db 1.pdf p.18~31, db 3.pdf p.14~15)

- **3) 3단계 실습: N:1 관계 판별과 스키마 설계**
  - *[1단계: 따라 하기]* 고객(`Customer`)과 주문(`Order`) 관계에서 한 고객은 여러 주문을 할 수 있으므로 `Order(N) : Customer(1)`이다. 외래 키 `customer_id`는 **`Order` 테이블**에 만든다.
  - *[2단계: 일부 변경하기]* 만약 은행(`Bank`)과 계좌(`Account`) 관계라면 어느 쪽이 N이고 어디에 FK를 두어야 하는가? → 한 은행에 여러 계좌가 개설되고 한 계좌는 한 은행에만 속하므로 **`Account(N) : Bank(1)`**이며, **`Account` 테이블**에 `bank_id`를 둔다 (`db 3.pdf` p.11~12).
  - *[3단계: 스스로 적용하기 (변형 문제)]* 편의점 프랜차이즈 시스템(`db 4.pdf` p.66 실습 1903번)에서 본사 브랜드(`Brand`), 가맹점(`Store`), 판매 상품(`Product`, 특정 가맹점에 등록됨) 세 테이블이 각각 1:N 관계일 때 외래 키 배치를 설계하시오.

정답 및 해설: 한 브랜드에 여러 가맹점이 속하므로 Store 모델에 brand = models.ForeignKey(Brand, ...)를 두고, 한 가맹점에 여러 상품이 등록되므로 Product 모델에 store = models.ForeignKey(Store, ...)를 둔다.

Unit A-2. 정수 나눗셈 버림(60000 vs 60000.0)과 ORDER BY 다중 정렬 · NULL 위치 [C05, C06]

- **1) 문제 상황 및 왜 어려운가**: ① Milliseconds / 60000을 계산했을 때 343719 / 60000 = 5.72865가 나올 것이라 예상하지만 실제로는 5가 출력된다. ② ORDER BY Country DESC, City;를 실행했을 때 City도 내림차순일 것이라 착각하거나, ORDER BY ReportsTo;에서 NULL이 맨 끝에 나올 것이라 오해한다.
- **2) 핵심 원리**: SQL에서 피연산자 두 개가 모두 정수(INTEGER / INTEGER)이면 **정수 나눗셈이 수행되어 소수점 이하가 완전히 잘려 나간다(Truncation)**. 소수점을 살리려면 피연산자 중 하나를 실수(60000.0)로 만들어야 한다. 또한 SQLite는 정렬 시 **NULL을 모든 숫자·문자보다 가장 작은 값으로 취급**하므로 오름차순(ASC) 정렬 시 맨 위에 배치한다.
- **3) 3대 핵심 검증 포인트 (아래 [그림 A-2]에서 확인할 흐름)**:
  - **자료형 승격(Type Promotion) 유무 비교**: 343719 / 60000은 반올림(6)이 아니라 소수점 이하 .72865가 통째로 절삭된 5가 되며, 60000.0을 나눌 때만 부동소수점 연산기(Float ALU)를 거쳐 5.72865가 보존된다.
  - **다중 컬럼 정렬의 독립성**: ORDER BY Country DESC, City에서 DESC는 직전 컬럼인 Country에만 적용된다. 1차 정렬에서 Country = 'USA'로 동률(Tie)이 발생한 그룹 내부에서만 2차 기준인 City ASC(기본값 오름차순: Cupertino → Mountain View → Reno)가 작동한다.
  - **SQLite NULL의 수직선 위치**: NULL < 음수 < 0 < 양수 < 문자열 순서로 평가되므로, ReportsTo ASC에서는 NULL이 1순위(맨 위), ReportsTo DESC에서는 NULL이 꼴찌(맨 아래)에 놓인다.



[그림 A-2] 정수/실수 나눗셈 타입 변환 비교, 다중 정렬 2단계 그룹 내부 정렬, SQLite NULL 대소 관계 수직선 (원본 db 1.pdf p.61~75)

- **3) 3단계 실습: 산술 나눗셈과 다중 정렬 · NULL 위치 예측**
  - *[1단계: 따라 하기]* tracks 테이블에서 Milliseconds 기준 내림차순으로 정렬하되 분 단위(정수)로 조회하시오: SELECT Name, Milliseconds / 60000 AS '재생 시간(분)' FROM tracks ORDER BY Milliseconds DESC;
  - *[2단계: 일부 변경하기]* 소수점까지 살아 있는 정확한 분 단위 값을 구하고, Composer 오름차순(1차) 및 재생 시간 내림차순(2차)으로 정렬하시오: SELECT Composer, Name, Milliseconds / 60000.0 AS min_val FROM tracks ORDER BY Composer ASC, min_val DESC; (이때 Composer가 NULL인 곡들이 맨 먼저 출력되며, 같은 NULL 그룹 안에서 재생 시간이 긴 곡부터 정렬된다!)
  - *[3단계: 스스로 적용하기]* 만약 ORDER BY ReportsTo DESC;를 실행하면 상사가 없는 사장(ReportsTo = NULL)은 첫 번째 줄에 나오는가, 마지막 줄에 나오는가? → SQLite는 NULL을 가장 작은 값으로 취급하므로 내림차순(DESC, 큰 값부터 작은 값 순서)에서는 **맨 마지막 줄**에 출력된다.

Unit A-3. SQL 3값 논리(TRUE / FALSE / UNKNOWN)와 와일드카드(%, _) · LIMIT OFFSET 슬라이싱 [C08, C09]

- **1) 문제 상황 및 왜 어려운가**: ① Company가 비어 있는 고객을 찾으려고 WHERE Company = NULL을 실행했는데 에러 메시지도 없이 결과만 0건이 나온다. ② LIMIT 4 OFFSET 3이 몇 번째부터 몇 번째 행을 가져오는지 매번 헷갈린다.
- **2) 핵심 원리**: NULL은 값이 아니라 "알 수 없음"이므로 NULL = NULL이나 NULL != 'USA'처럼 일반 비교 연산자(=, !=)를 쓰면 결과가 모두 **UNKNOWN**이 된다. WHERE 절은 오직 TRUE만 통과시키므로 UNKNOWN은 전부 버려진다. 따라서 반드시 **IS NULL / IS NOT NULL**을 써야 한다. 한편 OFFSET k는 **앞에서 k개를 버리고 (k+1)번째 행부터 시작**하라는 뜻이다.

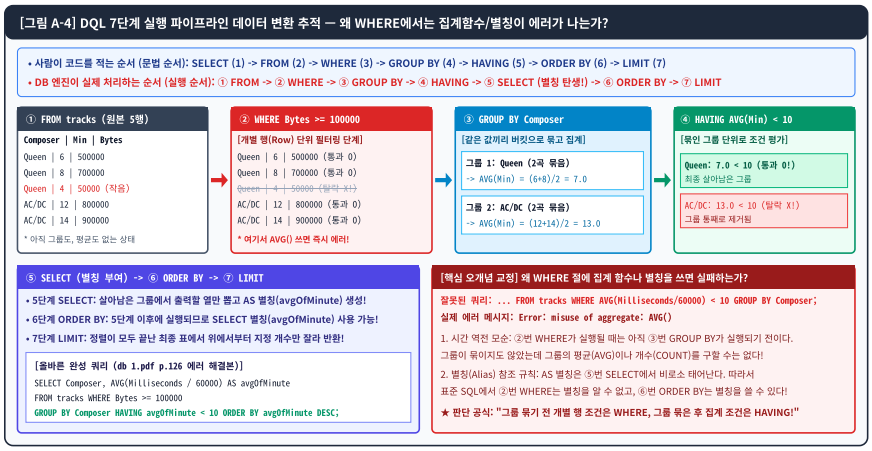


[그림 A-3] 3값 논리(UNKNOWN) 필터 게이트, 와일드카드 글자 슬롯 매칭, LIMIT 4 OFFSET 3 슬라이싱 (원본 db 1.pdf p.105~116, p.136~138)

- **3) 3단계 실습: 3값 논리(NULL) · 와일드카드 · LIMIT/OFFSET 조합**
  - *[1단계: 따라 하기]* customers 테이블에서 Company가 비어 있고(IS NULL) 국가가 'USA'인 고객을 조회하시오: `SELECT LastName, FirstName, Company, Country FROM customers WHERE Company IS NULL AND Country = 'USA';`
  - *[2단계: 일부 변경하기]* 성(LastName)이 'son'으로 끝나거나 이름(FirstName)이 정확히 4글자이면서 'a'로 끝나는 고객을 조회하시오: `SELECT LastName, FirstName FROM customers WHERE LastName LIKE '%son' OR FirstName LIKE '___a';`
  - *[3단계: 스스로 적용하기]* tracks 테이블에서 Bytes가 100,000 이상 500,000 이하인 곡들을 Bytes 내림차순으로 정렬한 뒤, **3번째부터 8번째까지의 곡(총 6곡)**만 조회하는 쿼리를 작성하시오.
    정답: `SELECT Name, Bytes FROM tracks WHERE Bytes BETWEEN 100000 AND 500000 ORDER BY Bytes DESC LIMIT 6 OFFSET 2;` ("3번째부터" → 앞의 2개 건너뜀 `OFFSET 2`, "3~8번째" → `8 - 3 + 1 = 6`개 `LIMIT 6`).

Unit A-4. DQL 7단계 실행 파이프라인과 WHERE vs HAVING 집계 에러 추적 [C10]

- **1) 문제 상황 및 왜 어려운가**: SQL 문을 작성할 때는 SELECT를 맨 먼저 적기 때문에, 학습자는 `SELECT AVG(Milliseconds / 60000) AS avgOfMinute`에서 만든 별칭과 집계값을 아래 줄의 `WHERE avgOfMinute < 10`에서 당연히 쓸 수 있다고 생각하여 **`misuse of aggregate: AVG()` 에러**를 겪는다(db 1.pdf p.126).
- **2) 핵심 원리**: SQL 엔진이 쿼리를 해석하고 실행하는 순서는 작성 순서와 완전히 다르다. **① FROM → ② WHERE → ③ GROUP BY → ④ HAVING → ⑤ SELECT → ⑥ ORDER BY → ⑦ LIMIT** 순서로 데이터가 파이프라인을 통과한다. 2단계(WHERE) 시점에는 아직 3단계(GROUP BY) 그룹핑도 안 되었고 5단계(SELECT) 별칭도 만들어지지 않았다!



[그림 A-4] DQL 7단계 실행 파이프라인에 따른 테이블 행·그룹 변환 추적 및 WHERE 집계 에러 발생 메커니즘 (원본 db 1.pdf p.126~131)

- **3) 3단계 실습: 그룹화(GROUP BY)와 조건절(WHERE + HAVING) 결합**
  - *[1단계: 따라 하기]* customers 테이블에서 국가(Country)별 고객 수를 조회하시오: `SELECT Country, COUNT(*) FROM customers GROUP BY Country;`
  - *[2단계: 일부 변경하기]* 위 쿼리에서 **회사(Company) 정보가 있는 고객들만 대상으로** 국가별 고객 수를 구하되, **고객 수가 5명 이상인 국가만** 고객 수 내림차순으로 조회하시오.

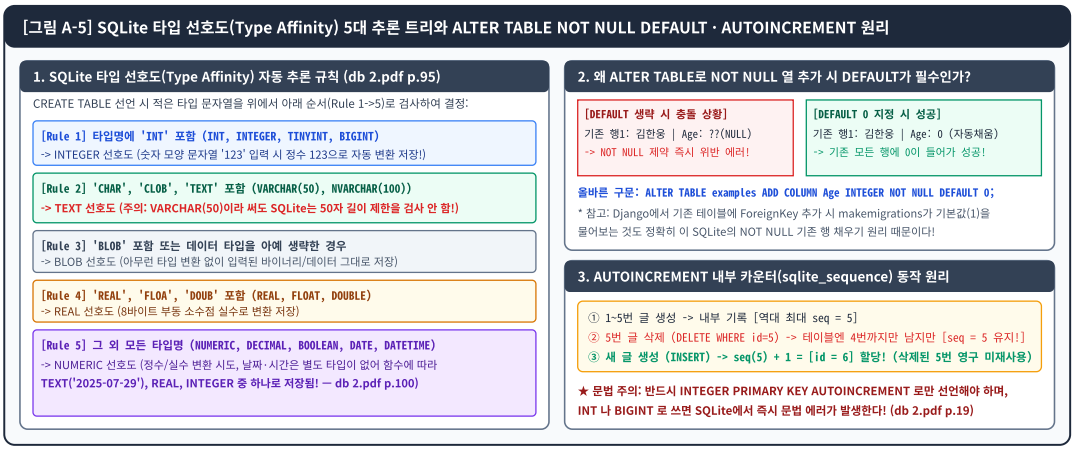
```
SELECT Country, COUNT(*) AS cnt
FROM customers
```

```
WHERE Company IS NOT NULL
GROUP BY Country
HAVING COUNT(*) >= 5
ORDER BY cnt DESC;
```

- *[3단계: 스스로 적용하기]* 만약 위 쿼리에서 `WHERE Company IS NOT NULL`을 빼고 `HAVING` 절에 넣거나, 반대로 `COUNT(*) >= 5`를 `WHERE`에 넣으면 결과가 어떻게 달라지는가? → 개별 고객의 `Company` 유무는 그룹화 전에 걸러야 정확한 집계가 되므로 `WHERE`에 써야 하고, 그룹별 인원수 조건은 `GROUP BY` 이후에만 알 수 있으므로 반드시 `HAVING`에 써야 한다.

Unit A-5. `AUTOINCREMENT` ID 비재사용 원리와 서브쿼리 결합 삭제(`DELETE WHERE id IN (...)`) [C11, C13]

- **1) 문제 상황 및 왜 어려운가**: ① `INTEGER PRIMARY KEY AUTOINCREMENT`가 설정된 테이블에서 1, 2, 3, 4, 5번 글을 넣고 5번 글을 지운 뒤 새 글을 넣었을 때, 번호가 5번이 될 것이라 예상하지만 실제로는 **6번**이 할당된다. ② "작성일이 가장 오래된 게시글 2개를 삭제하라"는 문제에서 `DELETE` 문에 바로 정렬과 개수 제한을 어떻게 결합해야 할지 막막해한다(db 2.pdf p.19, p.65~67).
- **2) 핵심 원리**: `AUTOINCREMENT`는 내부 시퀀스 테이블에 역대 최대값을 기록하여 **삭제된 값을 절대 재사용하지 않는다**(삭제된 회원이나 게시글의 ID가 다른 사람에게 재할당되면 과거 로그나 외래 키 참조가 엉키는 참사가 발생하기 때문). 또한 정렬 후 상위 N개를 지울 때는 **괄호 안의 서브쿼리(`SELECT id ... ORDER BY ... LIMIT N`)가 먼저 실행되어 ID 목록을 만들고, 바깥의 `DELETE ... WHERE id IN (...)`이 그 목록을 받아 삭제**한다.



[그림 A-5] SQLite 타입 선호도 5단계 추론 트리, ALTER TABLE NOT NULL DEFAULT 원리, AUTOINCREMENT 시퀀스 메모리 (원본 db 2.pdf p.19~27, p.95~100)

- **3) 3단계 실습: 테이블 생성 · 스키마 확인 · ALTER TABLE 제약 해결**
  - *[1단계: 따라 하기]* 자동 증가 기본 키를 가진 `examples` 테이블을 만들고 구조를 확인하시오: `CREATE TABLE examples (ExamId INTEGER PRIMARY KEY AUTOINCREMENT, LastName VARCHAR(50) NOT NULL); PRAGMA table_info('examples');`
  - *[2단계: 일부 변경하기]* `examples` 테이블에 `NOT NULL` 제약이 있는 `Country`와 `Age` 컬럼을 추가하시오: `ALTER TABLE examples ADD COLUMN Country VARCHAR(100) NOT NULL DEFAULT 'default value';` / `ALTER TABLE examples ADD COLUMN Age INTEGER NOT NULL DEFAULT 0;`
  - *[3단계: 스스로 적용하기]* 위 테이블에서 `Age` 컬럼 이름을 `UserAge`로 바꾸고, `Country` 컬럼을 삭제한 뒤, 테이블 이름을 `users_info`로 변경하시오.
    정답: `ALTER TABLE examples RENAME COLUMN Age TO UserAge;` → `ALTER TABLE examples DROP COLUMN Country;` → `ALTER TABLE examples RENAME TO users_info;`

Unit A-6. `INNER JOIN` vs `LEFT JOIN` 행 매칭 과정과 `LEFT JOIN + IS NULL` 안티조인 원리 [C14]

- **1) 문제 상황 및 왜 어려운가**: 단순히 벤 다이어그램(교집합 vs 왼쪽 원)만 외운 학습자는 실제 테이블에 **외래 키가 매칭되지 않는 행(4번 글: `userId = 4`)**이나 **참조당한 적 없는 부모 행(3번 회원: `'유하선'`)**이 섞여 있을 때 결과 행 개수와 `NULL`이 어느 컬럼에 생기는지 정확히 예측하지 못한다.
- **2) 핵심 원리**: `INNER JOIN`은 양쪽 테이블에 키가 모두 존재하는 행만 남기므로 4번 글과 3번 회원 유하선이 모두 사라진다. 반면 `FROM A LEFT JOIN B`는 **왼쪽 테이블(A)의 모든 행을 일단 결과표에 그대로 적어 놓고**, 오른쪽 테이블(B)에서 짝을 찾아 붙인다. 짝이 없으면 오른쪽 테이블에서 온 컬럼들을 전부 **`NULL`**로 채운다. 이 원리를 역이용해 `FROM users LEFT JOIN articles ... WHERE articles.userId IS NULL`을 실행하면 **오른쪽 칸이 `NULL`인 사람(= 글을 한 번도 안 쓴 회원 `'유하선'`)**만 정확히 걸러낼 수 있다!

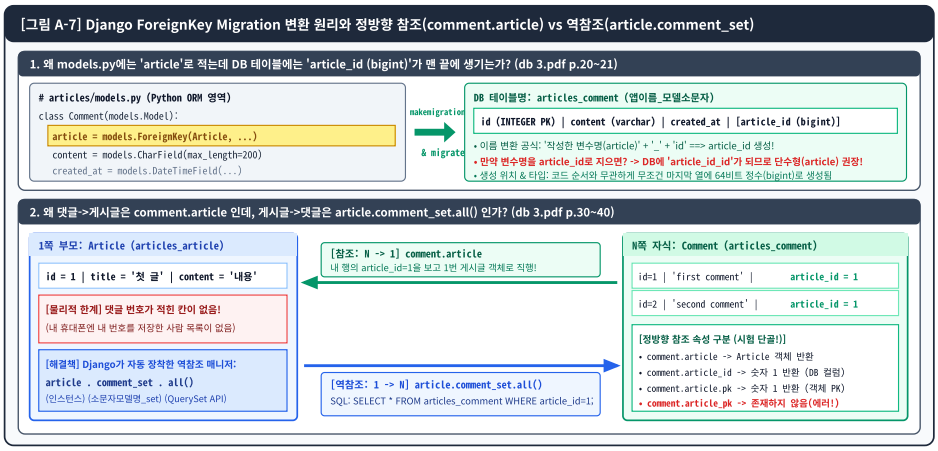


[그림 A-6] 서브쿼리 DELETE 실행 순서 및 INNER JOIN vs LEFT JOIN(articles 기준 vs users 기준 + IS NULL) 3대 비교 실험 (원본 db 2.pdf p.65~92)

- **3) 3단계 실습: 서브쿼리 삭제와 INNER JOIN vs LEFT JOIN 실전 구별**
  - *[1단계: 따라 하기]* 작성자가 실제로 존재하는 게시글만 제목과 작성자 이름을 조회하시오: `SELECT articles.title, users.name FROM articles INNER JOIN users ON users.id = articles.userId;`
  - *[2단계: 일부 변경하기]* 작성자 정보가 users 테이블에 없더라도(4번 글 userId=4) 모든 게시글을 빠짐없이 조회하시오: `SELECT articles.title, users.name FROM articles LEFT JOIN users ON users.id = articles.userId;`
  - *[3단계: 스스로 적용하기]* 반대로 **회원가입 후 게시글을 단 한 편도 작성하지 않은 회원의 이름**만 조회하시오: `SELECT users.name FROM users LEFT JOIN articles ON articles.userId = users.id WHERE articles.userId IS NULL;`

Unit A-7. Django ORM 정방향 참조(`comment.article`) vs 역참조(`article.comment_set.all()`) [C16]

- **1) 문제 상황 및 왜 어려운가**: Comment 모델에 `article = models.ForeignKey(Article, on_delete=models.CASCADE)`를 정의했을 때, 학습자는 `article.comments.all()`이나 `article.comment.all()`로 댓글을 가져오려다 `AttributeError`를 겪거나, `detail` 뷰에서 `Comment.objects.all()`을 써서 다른 게시글의 댓글까지 한꺼번에 출력되는 실수를 범한다(db 3.pdf p.31~38, p.56).
- **2) 핵심 원리**: DB의 `articles_comment` 테이블에는 `article_id` 컬럼이 있지만, `articles_article` 테이블에는 댓글에 대한 컬럼이 전혀 없다. 따라서 댓글에서 게시글로 갈 때는 내 행의 `article_id`를 읽는 **참조(`comment.article`)**를 쓰고, 게시글에서 댓글 목록을 찾을 때는 Django가 자동 생성해 준 역참조 매니저(모델소문자명_set)를 통한 **역참조(`article.comment_set.all()`)**를 써야 한다.

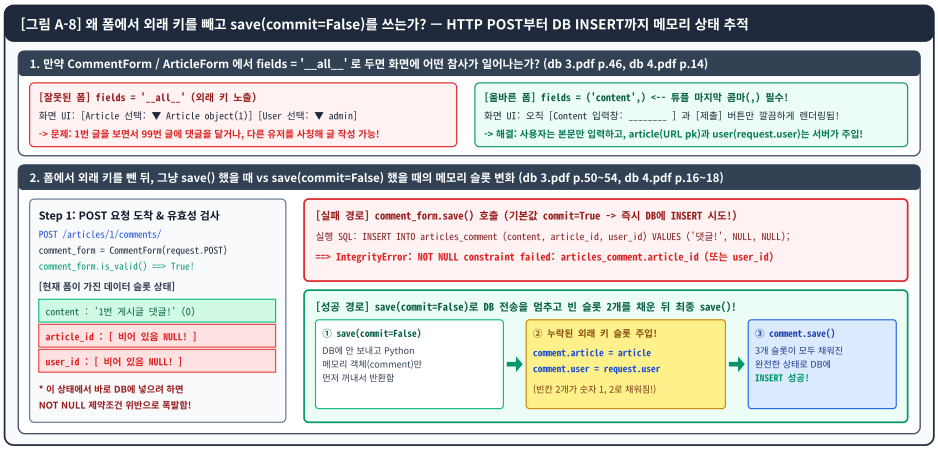


[그림 A-7] ForeignKey Migration DB 컬럼 변환 원리와 참조(N->1) vs 역참조(1->N) 양방향 탐색 구조도 (원본 db 3.pdf p.20~40)

- **3) 3단계 실습: 참조(`comment.article`) vs 역참조(`article.comment_set`) 코드 재현**
  - *[1단계: 따라 하기]* 1번 댓글 인스턴스 comment에서 자신이 속한 게시글의 PK와 본문 내용을 출력하시오: `print(comment.article.pk, comment.article.content)` (또는 `comment.article_id`).
  - *[2단계: 일부 변경하기]* 1번 게시글 인스턴스 article에 작성된 모든 댓글을 역참조하여 반복문으로 출력하시오: `for c in article.comment_set.all(): print(c.content)`
  - *[3단계: 스스로 적용하기]* 왜 `detail(request, pk)` 뷰 함수에서 `comments = Comment.objects.all()`을 쓰면 안 되는가? → `Comment.objects.all()`은 현재 보고 있는 게시글과 무관하게 DB의 모든 댓글을 다 가져오므로, 반드시 `article.comment_set.all()`을 써야 해당 글의 댓글만 조회된다.

Unit A-8. 왜 폼에서 외래 키를 제외하고 save(commit=False)로 주입하는가? [C17, C18]

- **1) 문제 상황 및 왜 어려운가**: CommentForm이나 ArticleForm에서 `fields = '__all__'`을 쓰면 화면에 Article이나 User를 고르는 선택창이 떠서 사칭·오작동이 발생한다. 그래서 `fields = ('content',)`로 외래 키를 뺐더니, 이번에는 `form.save()`를 호출할 때 **`IntegrityError: NOT NULL constraint failed`**가 발생하며 서버가 멈춘다.
- **2) 핵심 원리**: 사용자가 입력한 `request.POST` 안에는 오직 `content`만 들어 있고 `article_id`와 `user_id`는 비어 있다. 따라서 ① **`comment = comment_form.save(commit=False)`로 DB INSERT 요청을 잠시 멈추고 메모리상의 Comment 인스턴스만 먼저 꺼낸 뒤**, ② **URL Variable Routing으로 찾은 article과 로그인 세션의 request.user를 인스턴스 속성에 채워 넣고**, ③ **마지막에 `comment.save()`를 호출**해야 완벽한 레코드가 DB에 저장된다.

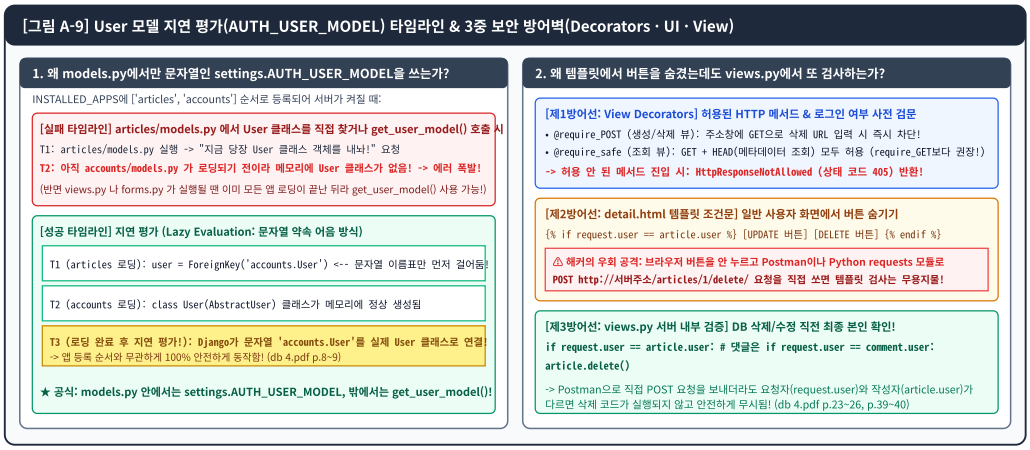


[그림 A-8] ModelForm 외래 키 제외 이유와 save(commit=False) 메모리 슬롯 주입 3단계 메커니즘 (원본 db 3.pdf p.43~54, db 4.pdf p.14~18)

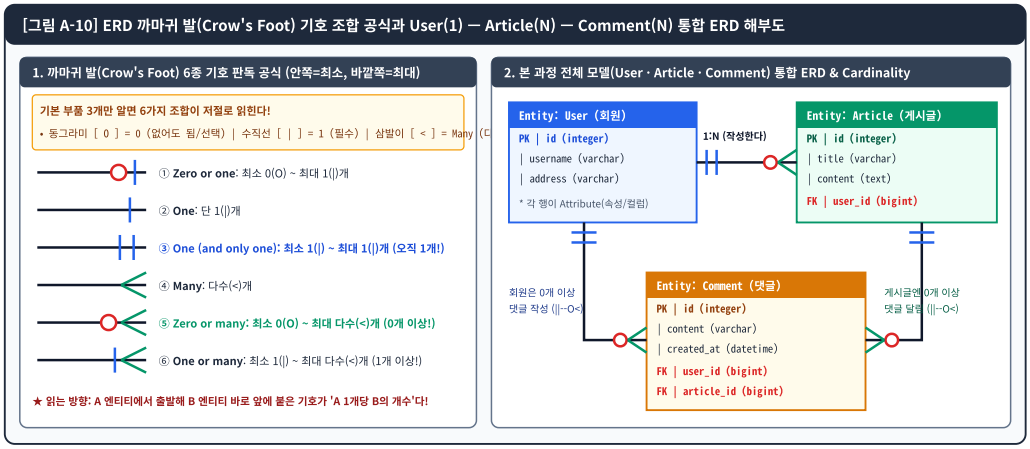
- **3) 3단계 실습: `CommentForm` 정의와 `save(commit=False)` 단계별 구현**
  - *[1단계: 따라 하기]* `forms.py`에서 외래 키를 제외하고 댓글 내용만 입력받는 폼을 정의하시오: `class CommentForm(forms.ModelForm): class Meta: model = Comment; fields = ('content',)`
  - *[2단계: 일부 변경하기]* `detail.html`에서 댓글 작성 폼의 action URL을 Variable Routing으로 연결하시오: `<form action="{% url 'articles:comments_create' article.pk %}" method="POST">`
  - *[3단계: 스스로 적용하기]* 만약 `comments_create` 뷰에서 `comment = comment_form.save(commit=False)`를 호출한 뒤 `comment.article = article`만 실행하고 마지막 `comment.save()`를 빼먹으면 어떤 결과가 발생하는가? → 에러는 발생하지 않지만 메모리에서만 객체가 수정되고 **실제 DB에는 INSERT 쿼리가 전송되지 않아 댓글이 저장되지 않는다!**

Unit A-9. AUTH_USER_MODEL 지연 평가, 이중 권한 제어, ERD Crow's Foot 통합 설계 [C18, C19, C20]

- **1) 문제 상황 및 왜 어려운가**: ① 왜 `models.py`에서만 유독 문자열을 반환하는 `settings.AUTH_USER_MODEL`을 쓰고 다른 곳에서는 `get_user_model()`을 쓰는지 이유를 모른 채 외운다. ② ERD의 까마귀 발(Crow's foot) 기호 6종이 나올 때마다 어느 쪽이 1이고 어느 쪽이 N인지 방향을 반대로 읽는다.
- **2) 핵심 원리**: Django가 구동될 때 `INSTALLED_APPS`에 등록된 앱들의 `models.py`를 순차적으로 읽는다. 만약 `articles/models.py`가 `accounts/models.py`보다 먼저 실행될 때 User 클래스를 직접 참조하면 아직 생성되지 않은 클래스를 찾게 되어 에러가 난다. 그래서 문자열 경로(`'accounts.User'`)를 먼저 적어 두고 모든 모델 로딩이 끝난 뒤 연결하는 **지연 평가(Lazy Evaluation)**를 사용한다. 또한 ERD의 기호는 **상대편 엔티티 바로 앞(선 끝부분)에 붙은 기호가 상대편의 개수(Cardinality)**를 뜻한다!
- **3) 3대 핵심 검증 포인트 (아래 [그림 A-9] · [그림 A-10]에서 확인할 흐름)**:
  - **앱 로딩 타임라인과 지연 평가(Lazy Evaluation)**: `INSTALLED_APPS`에서 `articles`가 `accounts`보다 위에 있을 때, `get_user_model()`은 즉시 객체 반환을 시도해 실패하지만 `settings.AUTH_USER_MODEL`은 문자열(`'accounts.User'`) 이름표만 걸어 두었다가 모든 앱의 `models.py` 로딩이 완료된 시점(T3)에 실제 클래스로 안전하게 연결된다.
  - **Postman 우회 공격과 3중 보안 방어벽**: 템플릿(`detail.html`)에서 `{% if request.user == article.user %}`로 삭제 버튼을 숨기는 것은 UI 편의 기능일 뿐이다. 악의적 사용자가 브라우저 밖에서 `POST /articles/1/delete/`를 직접 전송할 수 있으므로 반드시 뷰 함수 내부에서 데코레이터(`@require_POST`)와 작성자 일치 검사(`if request.user == article.user:`)를 이중 수행해야 한다.
  - **까마귀 발(Crow's Foot) 기호 분해 공식**: 안쪽 기호는 최소 개수(`0 = 0`, `| = 1`), 바깥쪽 기호는 최대 개수(`| = 1`, `< = Many`)를 뜻한다.



[그림 A-9] settings.AUTH_USER_MODEL 지연 평가 타임라인 및 Postman 우회 공격을 막는 3중 보안 아키텍처 (원본 db 4.pdf p.8~9, p.23~26, p.45~49)



[그림 A-10] ERD Crow's foot 최소·최대 기호 조합 원리 및 User·Article·Comment 3대 엔티티 통합 ERD (원본 db 4.pdf p.51~60)

- **3) 3단계 실습: User-Article-Comment 통합 구현 및 원본 실습 35개 대표 과제 해법**
  - *[1단계: 따라 하기]* `Comment` 모델에 게시글과 작성자를 모두 연결할 때:

    ```
    class Comment(models.Model):
        article = models.ForeignKey(Article, on_delete=models.CASCADE)
        user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)
        content = models.CharField(max_length=200)
    ```

  - *[2단계: 일부 변경하기]* 로그인한 사용자가 본인이 작성한 할 일(Todo) 목록만 모아보는 기능(`db 4.pdf` p.66 실습 3059번 '내 할 일 목록만 모아보기')을 역참조 또는 필터로 구현하시오.
    해법 1 (역참조): `todos = request.user.todo_set.all()`
    해법 2 (filter): `todos = Todo.objects.filter(user=request.user)`
  - *[3단계: 스스로 적용하기]* 댓글 삭제 뷰 함수 `comments_delete(request, article_pk, comment_pk)`에 로그인 확인, POST 전용 제한, 작성자 본인 확인을 모두 결합한 완성 코드를 작성하시오.

    ```
    @login_required
    @require_POST
    def comments_delete(request, article_pk, comment_pk):
        comment = Comment.objects.get(pk=comment_pk)
        if request.user == comment.user:
            comment.delete()
        return redirect('articles:detail', article_pk)
    ```

PART B. 외워두면 좋은 내용 (정확한 표기 · 사용 조건 · 흔한 실수 · 회상 연습)

PART B는 실전 코딩과 시험에서 작업 속도를 즉시 높여주는 필수 명령어·문법 패턴·단축 규칙을 [정확한 표기 → 사용하는 조건 → 실제 예시 및 결과 → 흔한 실수 → 회상 연습] 구조로 정리한 영역이다.

Unit B-1. SQL DQL · DDL · DML 필수 명령어 표기 및 조건 치트시트

구문 / 패턴명	정확한 문법 표기	사용 조건 및 결과	가장 흔한 실수 (오답 함정)
중복 제거 조회	`SELECT DISTINCT col FROM tbl;`	지정 컬럼의 중복 행 제거 후 고유값만 반환	`SELECT name, DISTINCT country`처럼 중간에 쓰면 문법 에러!
범위·목록 필터	`WHERE col BETWEEN a AND b` `WHERE col IN (v1, v2)`	a 이상 b 이하(경계 포함) / 목록 일치 확인	`IN` 대신 `OR` 쓸 때 컬럼명 생략(`WHERE c='A' OR 'B'`) 금지
패턴 매칭	`WHERE col LIKE '%son'` `WHERE col LIKE '___a'`	%(0글자 이상), _(정확히 1글자)	`= '%son'`으로 등호를 쓰면 와일드카드가 작동하지 않음!
NULL 비교	`WHERE col IS NULL` `WHERE col IS NOT NULL`	값이 비어 있는 행 / 값이 존재하는 행 추출	`= NULL` 또는 `!= NULL` 사용 시 UNKNOWN이 되어 0건 반환
구간 슬라이싱	`LIMIT row_count OFFSET offset;`	N번째~M번째: `OFFSET N-1, LIMIT M-N+1`	`LIMIT 3, 4`처럼 쉼표 축약형 사용 시 순서 혼동 위험
자동 증가 PK 선언	`id INTEGER PRIMARY KEY AUTOINCREMENT`	이전 최대값보다 큰 정수 자동 할당(재사용 X)	`INT PRIMARY KEY AUTOINCREMENT`로 쓰면 즉시 에러!
스키마 구조 확인	`PRAGMA table_info('tbl');`	`cid, name, type, notnull, dflt_value, pk` 출력	테이블명을 따옴표로 감싸고 세미콜론으로 마무리할 것
컬럼 추가 (NOT NULL)	`ALTER TABLE tbl ADD COLUMN col INT NOT NULL DEFAULT 0;`	기존 테이블에 NOT NULL 열 추가 시 기본값 부여	`DEFAULT` 생략 시 에러 / 한 문장으로 2개 컬럼 동시 추가 불가

컬럼·테이블 변경	`ALTER TABLE t RENAME COLUMN a TO b;`
	`ALTER TABLE t DROP COLUMN b;`
	`ALTER TABLE t RENAME TO t2;`
		컬럼명 변경 / 컬럼 삭제 / 테이블명 변경	전체 테이블 삭제는 `ALTER TABLE`이 아니라 `DROP TABLE t;`

서브쿼리 삭제	`DELETE FROM tbl WHERE id IN (SELECT id FROM tbl ORDER BY ... LIMIT n);`	정렬 기준 상위 n개 레코드 안전 삭제	`DELETE`나 `UPDATE`에서 `WHERE` 절 누락 시 전체 행 삭제/변경!

Unit B-2. Django Many-to-One (N:1) 구현 필수 코드 패턴 암기표

구현 단계	정확한 Python / DTL 코드 표기	적용 조건 및 흔한 실수 주의점
1. 게시글/댓글 ForeignKey 정의 (models.py)	`article = models.ForeignKey(Article, on_delete=models.CASCADE)` `user = models.ForeignKey(settings.AUTH_USER_MODEL, on_delete=models.CASCADE)`	• N쪽 모델에 소문자 단수형으로 선언 (DB에는 `article_id`, `user_id` 생성) • models.py에서는 절대 `get_user_model()`이나 User 직접 import 금지!

2. ModelForm 외래 키 제외 (forms.py)	`class CommentForm(forms.ModelForm):`
	`  class Meta:`
	`  model = Comment`
	`  fields = ('content',)`
		• 사칭 및 잘못된 게시글 선택 방지를 위해 `article`, `user` 제외
		• 1개 요소 튜플 작성 시 **끝에 콤마(,) 누락하면 에러 발생!**

구현 단계	정확한 Python / DTL 코드 표기	적용 조건 및 흔한 실수 주의점

3. 외래 키 주입 저장 (views.py)	`comment = comment_form.save(commit=False)`
	`comment.article = article`
	`comment.user = request.user`
	`comment.save()`
		• commit=False 없이 save() 호출 시 NOT NULL constraint failed 에러 발생
		• comment.article_pk라는 속성은 존재하지 않음!

4. 역참조 조회 및 템플릿 출력 (views.py / .html)	`comments = article.comment_set.all()`
	`{% for c in comments %} ... {% empty %} 댓글 없음 {% endfor %}`
	`댓글 수: {{ article.comment_set.count }}`
		• Comment.objects.all()을 쓰면 다른 글의 댓글까지 다 나옴!
		• related_name 미지정 시 기본 역참조 이름은 소문자모델명_set

5. 권한 검증 및 데코레이터 (views.py / .html)	`@login_required + @require_POST`
	`if request.user == comment.user:`
	`  comment.delete()`
		• 조회 뷰에는 @require_safe 권장 (GET + HEAD 허용)
		• 허용 안 된 메서드 요청 시 **405 Method Not Allowed** 반환

Unit B-3. 원본 교재 수록 전체 실습 과제(총 35개) 핵심 해결 공식 매핑표

원본 4권의 Run 실습 페이지(db 1.pdf p.141, db 2.pdf p.101, db 3.pdf p.71, db 4.pdf p.66)에 수록된 모든 실습 과제를 자습 중 혼자 재현할 수 있도록 핵심 해결 쿼리·코드 패턴을 매핑하였다.

원본 출처	실습 과제 번호 및 제목	혼자 재현하기 위한 핵심 SQL / Django 해결 공식
SQL 01 (db 1 p.141)	• 1888. SQL 조회 - 기초 • 2899. 회원 목록 DB - 조회 • 3050. SQL조회, 정렬 • 2900. 데이터 정렬과 중복제거	• 전체/특정 열: `SELECT * FROM ...;` / `SELECT name, age FROM users;` • 다중 정렬: `ORDER BY age ASC, balance DESC;` • 중복 제거 후 정렬: `SELECT DISTINCT country FROM users ORDER BY country;`
SQL 01 (db 1 p.141)	• 2901. 회원 목록 DB - WildCards • 2903. 조건부 조회 심화 • 1889 / 3051 / 2902. 그룹화·집계	• 전화번호/이름 패턴: `WHERE phone LIKE '02-%'`, `WHERE first_name LIKE '___'` • 그룹별 집계와 필터: `SELECT country, AVG(balance) FROM users GROUP BY country HAVING AVG(balance) >= 1000 ORDER BY AVG(balance) DESC;`
SQL 02 (db 2 p.101)	• 1894. 테이블 생성과 데이터 삽입 • 1895. 테이블 삭제와 JOIN • 3052 / 3053 / 3054. artists, invoices	• `CREATE TABLE orders (id INTEGER PRIMARY KEY AUTOINCREMENT, ...);` • `SELECT * FROM orders INNER JOIN customers ON customers.id = orders.customer_id;` • 다중 조건 조회: `WHERE BillingCountry = 'USA' AND Total >= 10`
SQL 02 (db 2 p.101)	• 2906 / 2907. 동물원 DB 관리 • 2908. 호텔 객실 DB 관리 • 2909. 유저 정보 / 3278. 직원 집계	• `ALTER TABLE zoo ADD COLUMN species TEXT NOT NULL DEFAULT 'unknown';` • 조건부 수정/삭제: `UPDATE hotels SET grade = 'VIP' WHERE room_num >= 1000;` • `DELETE FROM animals WHERE weight < 10;`
M:1 01 (db 3 p.71)	• 1901 / 1902. 도서 관리 서비스 (1:N) • 1896 / 1898 / 1899 / 1900. 레스토랑 • 3055 / 3056 / 3057. 댓글 생성·조회·삭제	• 도서(Book, N)가 작가(Author, 1) 참조: `author = models.ForeignKey(Author, on_delete=models.CASCADE)` → 작가 상세에서 `author.book_set.all()` 조회 • 메뉴(Menu, N)가 식당(Restaurant, 1) 참조: `menu = form.save(commit=False); menu.restaurant = restaurant; menu.save()`

원본 출처	실습 과제 번호 및 제목	혼자 재현하기 위한 핵심 SQL / Django 해결 공식
M:1 02 (db 4 p.66)	• 3058 / 3059. 할 일 목록 작성자·모아보기 • 1908 / 1909. 리뷰 서비스와 유저 권한 • 1903~1906. 편의점 프랜차이즈·auth	• 작성자 저장: `todo.user = request.user` / 내 글만: `request.user.todo_set.all()` • 리뷰 삭제 권한: `@login_required` + `if request.user == review.user: review.delete()`

Unit B-4. 즉각 인출을 위한 빈칸 회상 훈련 (Active Recall Drill 10문항)

1. SQLite에서 정수 나눗셈 버림을 막고 분 단위 소수점을 유지하려면 `Milliseconds / (  )`으로 나눈다.
2. 정렬된 결과에서 앞의 3개를 건너뛰고 4번째부터 7번째(총 4개)만 가져오는 절은 `LIMIT (  ) OFFSET (  )`이다.
3. SELECT 7단계 실행 순서는 `FROM` → ( ) → `GROUP BY` → ( ) → `SELECT` → ( ) → `LIMIT`이다.
4. SQLite에서 자동 증가 기본 키는 반드시 ( ) `PRIMARY KEY AUTOINCREMENT`로 선언해야 한다.
5. 게시글을 한 번도 쓰지 않은 회원을 찾을 때는 `FROM users (  ) JOIN articles ... WHERE articles.userId (  )`을 쓴다.
6. Django에서 `Comment`가 `Article`을 참조할 때 DB에 자동 생성되는 외래 키 컬럼명은 ( )이고 자료형은 ( )이다.
7. 1번 게시글 `article`에 달린 모든 댓글을 역참조하는 기본 코드는 `article.(  ).all()`이다.
8. 폼 데이터를 DB에 즉시 저장하지 않고 인스턴스만 반환받는 메서드는 `form.save((  ))`이다.
9. `models.py` 내부에서 User 모델을 참조할 때는 ( )을 쓰고, 그 외 파일에서는 ( )을 쓴다.
10. 뷰 데코레이터에서 GET과 HEAD를 모두 허용하는 권장 데코레이터는 `@(  )`이며, 위반 시 상태 코드 ( )를 반환한다.

[회상 훈련 정답] 1) `60000.0` | 2) `4`, `3` | 3) `WHERE`, `HAVING`, `ORDER BY` | 4) `INTEGER` | 5) `LEFT`, `IS NULL` | 6) `article_id`, `bigint` | 7) `comment_set` | 8) `commit=False` | 9) `settings.AUTH_USER_MODEL`, `get_user_model()` | 10) `require_safe`, `405`